

AI Code Intelligence & Code Forensics Platform

Final-Year Academic Project — 30-Day Execution Blueprint

Mentor-approved objective: Build a technically strong, placement-oriented working prototype within approximately 100 hours, while understanding every major component.

1. Project Goal

The platform analyzes Python source code and produces a structured intelligence report covering code quality, bug risk, security findings, and a computational Code DNA fingerprint. An AI explanation layer converts the detected evidence into understandable recommendations, and a dashboard presents the results.

2. Final Workflow

Stage	What happens
1. Source Code	User uploads a Python file or project.
2. Parser / AST	The system converts source code into a structural representation.
3. Feature Extraction	Extract functions, classes, imports, branches, loops, calls, nesting and other features.
4. Code Quality	Calculate code metrics and quality indicators.
5. Bug-Risk Prediction	ML model estimates defect risk from extracted features.
6. Security Analysis	Rule-based/static checks identify recognizable unsafe patterns.
7. Code DNA	Build a coding-style feature vector and compare code samples.
8. AI Explanation	An AI layer explains findings using evidence produced by the analysis engine.
9. Dashboard	Present scores, findings, fingerprint and explanations interactively.

3. Core Modules

Module	Purpose	Priority
Code Intelligence	Parse code and extract structural/quality features.	Must have
Bug-Risk Prediction	Predict Low/Medium/High defect risk using ML.	Must have
Security Analysis	Detect predefined suspicious/unsafe coding patterns.	Must have
Code DNA	Represent coding characteristics as a computational fingerprint.	Signature feature
AI Explanation	Explain findings and recommendations from collected evidence.	Must have
Dashboard	Provide a polished interface for the complete analysis.	Must have

4. 30-Day Plan — 25 Hours/Week

Available time: Weekdays maximum 3 hours/day; Saturday and Sunday 5 hours/day. Total target $\approx$ 25 hours/week and $\approx$ 100 hours across four weeks.

WEEK 1 — Foundation & Code Intelligence

Mon — 3h — Understand source-code analysis, AST, static analysis, code forensics, Code DNA.

Tue — 3h — Set up Python environment, VS Code, Git, virtual environment and repository structure.

Wed — 3h — Learn AST: nodes, functions, classes, loops and conditions.

Thu — 3h — Build AST parser and extract functions, classes, imports, loops, conditions and calls.

Fri — 3h — Implement basic metrics: LOC, functions, nesting, function length and complexity indicators.

Sat — 5h — Integrate: Python file $\rightarrow$ AST $\rightarrow$ features $\rightarrow$ metrics $\rightarrow$ JSON/report.

Sun — 5h — Test using multiple Python programs; fix parser and feature-extraction issues.

WEEK 2 — Bug-Risk ML

Mon — 3h — Understand software defects, classification, features, labels and bug-risk prediction.

Tue — 3h — Select and understand a suitable software-defect dataset; clean and inspect it.

Wed — 3h — Train a baseline model such as Logistic Regression.

Thu — 3h — Train a second model such as Random Forest and compare.

Fri — 3h — Learn and calculate accuracy, precision, recall, F1-score and confusion matrix.

Sat — 5h — Connect extracted code features to the ML prediction pipeline.

Sun — 5h — Test, evaluate, document results and improve the pipeline.

WEEK 3 — Security + Code DNA

Mon — 3h — Learn static security analysis and recognizable unsafe coding patterns.

Tue — 3h — Implement security rules and produce finding, location, severity and explanation.

Wed — 3h — Learn Code DNA: coding fingerprints, feature vectors and similarity.

Thu — 3h — Extract developer-style features such as naming, structure and complexity tendencies.

Fri — 3h — Implement fingerprint/similarity comparison.

Sat — 5h — Integrate quality + bug risk + security + Code DNA.

Sun — 5h — Test with different code samples and coding styles.

WEEK 4 — AI + Dashboard + Finalization

Mon — 3h — Design AI explanation layer. AI receives evidence; it does not replace the analysis engine.

Tue — 3h — Implement evidence-grounded explanations.

Wed — 3h — Build dashboard layout and result views.

Thu — 3h — Integrate all modules.

Fri — 3h — End-to-end testing and debugging.

Sat — 5h — Polish UI, charts, reports, error handling and documentation.

Sun — 5h — Prepare report, PPT, architecture, results, screenshots, demo flow and viva questions.

5. Weekly Milestones

Week	Must-have outcome
1	A Python file can be parsed and converted into structured features and a basic code-quality report.
2	A trained ML model can estimate bug risk and its performance is evaluated properly.
3	Security findings and Code DNA fingerprint/similarity are integrated with the analysis engine.
4	AI explanation, dashboard, end-to-end testing, evaluation and academic deliverables are complete.

6. Daily 3-Hour Routine

Time	Activity
30 min	Learn the concept needed for the day's task.
1h 45 min	Implement the planned feature.
30 min	Test and debug.
15 min	Write a short progress note: what was learned, what works, what failed.

7. Proposed Technology Areas

Primary language: Python

Core analysis: Python AST and static-analysis techniques

ML: scikit-learn-style workflow; final model chosen after dataset evaluation

Data: tabular software-defect dataset + our own test-code collection

AI explanation: LLM/API only after deterministic analysis is working

Interface: lightweight Python web dashboard

Version control: Git/GitHub

Important: exact libraries will be frozen during setup so the one-month scope remains stable.

8. Evaluation Plan

We will not claim success merely because the demo works. The project will include model metrics such as accuracy, precision, recall and F1-score where appropriate; confusion matrices for classification; examples of correct and incorrect analyses; security test cases; Code DNA similarity experiments; and an explicit limitations section.

9. Strict Mentor Rules

1. Do not add unrelated features during the month.
2. Do not copy code that you cannot explain.
3. Do not call an LLM API and treat that alone as the AI system.
4. Do not claim bug detection when the model only predicts risk.
5. Do not claim Code DNA proves a person's identity; it is a computational fingerprint/similarity mechanism.
6. Every week must produce something runnable.
7. If a task is unfinished, report it honestly and we adjust the schedule.
8. College work and exams take priority; the project schedule can be shifted rather than forcing unhealthy workload.

10. Final Demo Flow

1. Open dashboard → 2. Upload/select Python source → 3. Parse and extract features → 4. Show quality metrics → 5. Show bug-risk prediction + evaluation evidence → 6. Show security findings → 7. Show Code DNA fingerprint and similarity → 8. Generate evidence-grounded AI explanation → 9. Show final report.

11. Final Academic Deliverables

- Working source code and reproducible environment
- Dataset description and preprocessing
- System architecture diagram
- Module descriptions
- ML methodology and evaluation
- Security-analysis test cases
- Code DNA methodology and limitations
- Dashboard screenshots
- Final results and limitations
- Project report
- Presentation (PPT)
- Viva preparation and demo script

Mentor principle: A finished, tested and understood 8/10 project is better than an unfinished 10/10 idea. Our one-month strategy is therefore scope discipline, incremental milestones, evidence-based evaluation and strong understanding.